Full-Stack JavaScript Developer Test

Purpose

This assessment evaluates a candidate's full-stack development skills, software architecture decisions, API design, database modeling, authentication implementation, and ability to build scalable production-ready applications.

1. General Requirements

- Back-End must be developed using NestJS or any NodeJs framework with using Typescript. (Using NestJs is recommended)
- Front-End must be developed using a modern framework/library (React or NextJs).
- Use PostgreSQL as the primary database.
- Implement authentication using JWT or a comparable token-based solution.
- Validate all request bodies and query parameters.
- Use a well-organized folder structure.
- Write clear and maintainable code with comments where appropriate.
- Deploy both Front-End and Back-End to a cloud provider (AWS, Azure, GCP, etc.).
- Apply a mobile-first responsive design approach.
- Use a UI component library (Tailwind CSS, Ant Design, Material UI, Bootstrap, etc.).
- Use at least two different HTTP methods.
- Provide a comprehensive README.
- Use of AI-assisted development tools is allowed and encouraged.

2. Functional Requirements

Question 1 – Game Listing

Create a page displaying all games from **game-data.json**. Game data must be served by the Back-End through a REST API endpoint. Use thumb.url to display thumbnails.

Question 2 – Search Functionality

Implement a search feature that filters games while the user types. Filtering must be performed on the Back-End through a dedicated REST endpoint.

Question 3 – Authentication & Slot Machine

Implement a complete authentication system including:

• User Registration
• User Login
• JWT Authentication
• Protected Endpoints

The slot machine functionality must only be accessible to authenticated users.

Each newly registered user starts with 20 coins.

Every spin:
• Costs between 0.50 and 5.00 coins per spin (user-selectable)
• Updates the user's balance
• Must be stored permanently in the database

The spin record must include:
• User ID
• Game ID (if applicable)
• Reel results
• Amount won/lost
• Balance before spin
• Balance after spin
• Timestamp

The API response must return:
• Reel result
• Win/Loss amount
• Updated balance
• Spin identifier

Slot Machine Rules & Winning Logic

The slot machine consists of three reels with fixed symbols:

Reel 1: ["cherry", "lemon", "apple", "lemon", "banana", "banana", "lemon", "lemon"]

Reel 2: ["lemon", "apple", "lemon", "lemon", "cherry", "apple", "banana", "lemon"]

Reel 3: ["lemon", "apple", "lemon", "apple", "cherry", "lemon", "banana", "lemon"]

Spin Behaviour

• For each spin, the system must randomly select one symbol from each reel.

• The selected symbols form the final spin result in left-to-right order (Reel 1 → Reel 2 → Reel 3).

• The selected spin amount must be deducted from the user's balance before calculating winnings.

• If the user does not have sufficient balance, the spin must be rejected.

Winning Rules

• 3 cherries in a row: Spin Cost × 50

• 2 cherries in a row: Spin Cost × 40

• 3 apples in a row: Spin Cost × 20

• 2 apples in a row: Spin Cost × 10

• 3 bananas in a row: Spin Cost × 15

• 2 bananas in a row: Spin Cost × 5

• 3 lemons in a row: Spin Cost × 3

Matching Rules

• A match is only valid when symbols appear consecutively from left to right, starting with Reel 1.

Examples:

• Apple, Cherry, Apple – No win

• Apple, Apple, Cherry – Win (2 apples)

• Cherry, Cherry, Lemon – Win (2 cherries)

• Banana, Banana, Banana – Win (3 bananas)

• Lemon, Lemon, Lemon – Win (3 lemons)

• Lemon, Lemon, Apple – No win

Only the highest applicable payout for a single spin should be awarded. Payouts are not cumulative.

Balance Update

New Balance = Previous Balance – Spin Amount + Winnings

The complete spin transaction must be persisted in the database together with the fields defined in Question 3.

• The user must be able to select a spin amount using a Front-End control (e.g., dropdown, segmented selector, slider, or similar UI component). Available bet amounts must range from 0.50 to 5.00 coins in increments of 0.50 coins (inclusive).

Question 4 – Middleware & Security

Use any middleware necessary to improve API robustness and security. Examples include rate limiting, authentication guards, request validation, logging, security headers, and error handling. Clearly mark the implementation.

Question 5 – Search Optimization

Optimize the search endpoint to reduce server load. Examples include debouncing, caching, indexing, pagination, throttling, query optimization, or other scalable approaches. Clearly document the solution.

Question 6 – Currency Conversion

Add a button that converts the current balance into another currency using an external exchange-rate API. Conversion is for display purposes only and must not modify the stored balance.

Question 7 – Database Design

Design a relational database schema for an online casino platform.

Required entities:
- Users
- Games

- Game Types
- Countries
- User Favorite Games
- Spin History

Requirements:
- A casino contains multiple games.
- Each game has a unique type.
- Games can be available in multiple countries.
- Users may have a favorite game.
- Every spin must be recorded permanently.
- Relationships must be normalized.

Provide:
- ER Diagram
- SQL CREATE TABLE statements
- Primary Keys, Foreign Keys, Indexes, and Constraints

Question 8 – AI Usage Disclosure

If AI tools were used, explain in detail how they were used, which tasks they assisted with, and how generated code was reviewed or modified.

3. Evaluation Criteria

- Code quality and architecture
- Database design
- Authentication and security
- API design
- Scalability considerations
- UI/UX quality
- Documentation quality
- Testing approach